

A small horizontal bar with a teal segment on the left and an orange segment on the right.

CSC 84040: Advanced Data Mining *with AI Agents*

Lecture 04: Data Streaming

By Saptarashmi Bandyopadhyay

Email: sbandyopadhyay@ccny.cuny.edu

Assistant Professor of Computer Science

City College of New York and Graduate Center at the City University of New York

September 23, 2026 CSC 84040



Why do we need Data Streams?

In database systems, data sits statically on disk waiting to be queried. In data stream mining, data arrives unprompted, rapidly, and continuously, if not processed immediately, it is lost.

We need to try to work with an unbounded stream ($N \rightarrow \infty$) under strictly bounded memory ($M \ll N$)

Lecture Scope:

- Real-time operating constraints: Lock-free ingestion and deterministic execution.
- Core sketching primitives: Bloom filters, Flajolet-Martin, Count-Min sketches.
- Windowed stream models: DGIM sliding-window counter.

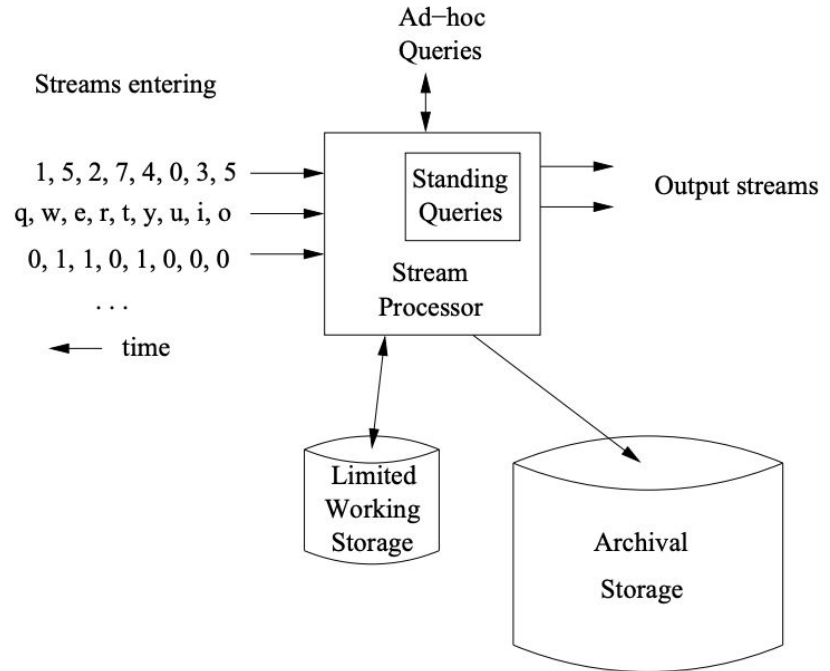
Stream Management

System Breakdown:

- **Input Streams:** Multiple, asynchronous arrival schedules with non-uniform rates.
- **Working Store:** High-speed, capacity-limited memory for active query execution.
- **Archival Store:** High-capacity secondary storage; infeasible for direct, ad-hoc queries.

Types of queries:

- Evaluating incoming elements against permanently active filters on-the-fly (**Standing Queries**)
- Using summaries from working memory to answer questions on an as-needed basis (**Ad-Hoc Queries**)





Case Study: Real-Time Constraints in Robotics

High-Rate Sampling:

- Needed in high-fidelity sensor data (e.g. IMU): 200 Hz — 1 kHz (1 ms processing deadline).
- Motor thermal sub-streams: 10 — 50 Hz.

Determinism:

- Standard OS kernels introduce non-deterministic latency jitter via page faults and preemption.
- Heap allocations (`malloc`, dynamic vector reallocations) during ingestion violate real-time deadlines.



Lock-Free Ring Buffers

Architectural Separation:

- **Ingestion Thread (Hard RT):** Reads hardware registers via memory-mapped I/O; never blocks on a mutex; writes to atomic ring buffer.
- **Consumer Thread (Non-RT):** Reads from the ring buffer; executes network streaming, serialization, or disk writes.

Concurrency Mechanics:

- Lock-free synchronization via `std::atomic` head and tail indices with `acquire-release` memory ordering.
- Eliminates priority inversion caused by standard mutex locks.



Representative Stream Sampling

Why we can't use uniform sampling:

- Sampling 1/10th of incoming search engine queries uniformly at random distorts duplicate-query statistics.
- For queries appearing twice, the probability both are sampled is $(1/10)^2 = 1/100$, severely underestimating user repeat frequency.

Hash-Partitioned Key Sampling:

- Let a stream element be $(user, query, time)$.
- Define a hash function $h : user \rightarrow [0, B - 1]$.
- Accept tuple into sample if $h(user) < a$, yielding an exact a/B sample of users while preserving all of their individual queries.
- **Dynamic Budgeting:** If memory fills, lower threshold $t \leftarrow t - 1$ and evict all entries where $h(key) = t$.



Bloom Filter for Stream Filtering

We want to be able to use a whitelist S which has m elements when the size of $|S|$ maxes out available RAM.

Structure:

- A bit array B of size n , initialized to all zeroes.
- k independent, uniform hash functions $h_1, h_2, \dots, h_k : S \rightarrow [0, n - 1]$.

Operations:

- **Insertion:** For each $x \in S$, set $B[h_i(x)] \leftarrow 1$ for all $1 \leq i \leq k$.
- **Query:** For incoming element y , evaluate $\bigwedge_{i=1}^k B[h_i(y)]$.
- If any $B[h_i(y)] == 0 \implies y \notin S$ (Strict zero false negatives).
- If all $B[h_i(y)] == 1 \implies y \in S$ with a false positive probability ϵ .



Bloom Filter Error Prob. and Sizing

Derivation of False Positive Probability:

- Probability a specific bit remains 0 after inserting m keys with k hash functions:

$$P[\text{bit is 0}] = \left(1 - \frac{1}{n}\right)^{km} \approx e^{-km/n}$$

- Probability of a false positive on an absent key:

$$p = \left(1 - e^{-km/n}\right)^k$$

Optimal Parameter Sizing:

- If we minimize p w/rt k considering a fixed n and m , we get:

$$k = \frac{n}{m} \ln 2 \approx 0.693 \frac{n}{m}$$

- Optimal array length for desired false positive rate p :

$$n = -\frac{m \ln p}{(\ln 2)^2}$$



Distinct Element Counting using Flajolet-Martin

Suppose we want to estimate the number of unique elements F_0 in an unbounded stream without storing seen keys.

Tail-Length Intuition:

- Hash each element a to a 64-bit integer: $h(a)$.
- Let $\rho(h(a))$ be the number of trailing zeros in the binary representation of $h(a)$.
- $P[\rho(h(a)) \geq r] = 2^{-r}$.
- If we observe a maximum tail length $R = \max \rho(h(a))$, the estimate for distinct elements is 2^R .

Combining Estimators:

- Taking a raw arithmetic mean yields infinite variance due to outlier tail values.
- Taking a pure median forces estimates into powers of 2.
- To avoid this, we can partition hash functions into small groups, take the arithmetic average within each group, and compute the median across group averages.

Frequency Moments of a Stream

Definition: Let m_i be the occurrence count of element i from a universe of items. The k -th order frequency moment is:

$$F_k = \sum_{i=1}^n m_i^k$$

Interpretations Across Orders:

- F_0 : Count of distinct elements ($\sum m_i^0$ where $m_i > 0$).
- F_1 : Total length of the stream ($\sum m_i$).
- F_2 : Measures non-uniformity and variance of stream frequencies (aka **The Surprise Number** 🎉)

X-treme Distributions ($N = 100$, 11 distinct items):

- Flat/Uniform (1 item 10 times, 10 items 9 times): $F_2 = 10^2 + 10(9^2) = 910$
- Skewed (1 item 90 times, 10 items 1 time): $F_2 = 90^2 + 10(1^2) = 8,110$

Estimating F_2 : Alon-Matias-Szegedy (AMS)

The AMS Estimator:

- Sample a random position in a stream of length n .
- Track element X .element and maintain its running frequency from that point forward: $v = X$.value.
- Compute estimate: $e_X = n(2v - 1)$.

Expectation Proof:

$$E[e_X] = \frac{1}{n} \sum_{i=1}^n n(2c(i) - 1) = \sum_a \sum_{v=1}^{m_a} (2v - 1)$$

- Since $\sum_{v=1}^{m_a} (2v - 1) = m_a^2$, we have $E[e_X] = \sum_a m_a^2 = F_2$

Handling unbounded streams with the “Reservoir Technique”

- With capacity for s trackers, retain first s items.
- For item $n + 1$, sample with probability $s/(n + 1)$; replace an existing tracker chosen uniformly at random.
- Guarantees all historical positions maintain equal probability s/n .



Count-Min Sketch Architecture

Data Structure Parameters:

- 2D counter array C of depth d and width w : $w = \left\lceil \frac{e}{\epsilon} \right\rceil$, $d = \left\lceil \ln \left(\frac{1}{\delta} \right) \right\rceil$
- d pairwise-independent hash functions $h_j : \{1 \dots n\} \rightarrow \{1 \dots w\}$

Stream Update (i_t, c_t) :

- For each row $j \in \{1 \dots d\}$: $C[j, h_j(i_t)] \leftarrow C[j, h_j(i_t)] + c_t$
- Memory: $O\left(\frac{1}{\epsilon} \ln \frac{1}{\delta}\right)$ words. Update time: $O\left(\ln \frac{1}{\delta}\right)$.



Point Queries & Heavy Hitters via CM Sketch

Point Query Estimation ($\hat{a}_i$):

$$\hat{a}_i = \min_{1 \leq j \leq d} C[j, h_j(i)]$$

One-Sided Error Guarantee:

$$a_i \leq \hat{a}_i \quad \text{and} \quad P[\hat{a}_i \leq a_i + \epsilon \|a\|_1] \geq 1 - \delta$$

Proof Sketch (Markov's Inequality):

- Indicator $I_{i,j,k} = 1$ if $h_j(i) = h_j(k)$ ($k \neq i$).
- $C[j, h_j(i)] = a_i + X_{i,j}$, where $E[X_{i,j}] \leq \frac{\epsilon}{e} \|a\|_1$.
- $P[C[j, h_j(i)] - a_i \geq \epsilon \|a\|_1] = P[X_{i,j} \geq e E[X_{i,j}]] \leq e^{-1}$
- Across d independent rows: $P[\hat{a}_i - a_i \geq \epsilon \|a\|_1] \leq e^{-d} \leq \delta$.

Heavy Hitters: Maintain candidate elements in a min-heap; evict if $\hat{a}_i < \phi \|a\|_1$.



Online Quantile Estimation

The Core Mechanism:

- Clusters streaming points into ordered centroids with mean μ_i and weight w_i .
- Enforces a non-linear scale function: centroid sizes remain small near tails ($q \rightarrow 0$ and $q \rightarrow 1$) and grow larger in the middle.

Operational Advantages:

- Extremely high accuracy for extreme percentiles (p99, p99.9, p99.99) with sublinear memory.
- **Combine-ability:** t -digests computed over parallel windows or distributed nodes can be summed with minimal loss in accuracy.



Sliding Windows: The Impossibility of Exact Counting

Given a bitstream, we want to answer: "How many 1s are in the last k bits?" for any $k \leq N$.

Space Lower Bound Theorem:

- An exact count of 1s in a window of size N requires $\Omega(N)$ bits of storage.

Proof by Pigeonhole Principle:

- There are 2^N possible binary sequences of length N .
- Suppose an algorithm uses $< N$ bits; then two distinct bitstrings $x \neq y$ must map to the same internal state.
- Let x and y disagree at the k -th position from the right.
- If queried for the last k bits, the algorithm must output the same value for both, but the ground truth counts differ by 1.
- Contradiction: Exact counting requires storing all N bits.



DGIM (Datar-Gionis-Indyk-Motwani) Algorithm

- **Storage Reduction:** Represents window of size N using only $O(\log^2 N)$ bits.
- **The Six Invariant Rules:**
 1. The right end of every bucket is a 1.
 2. Every 1 belongs to a bucket.
 3. No bit is assigned to multiple buckets.
 4. Bucket sizes are powers of 2 (1, 2, 4, 8, ...).
 5. There are **either 1 or 2 buckets** of any given size.
 6. Bucket sizes non-decrease moving backward in time (leftward).



DGIM Maintenance & Ripple Merging

Update Procedure upon Bit Arrival:

- **Expiry:** If oldest bucket timestamp $\leq t_{\text{current}} - N$, drop the bucket.
- If bit is 0: Take no action.
- If bit is 1:
 - Instantiate a new bucket: [size: 1, timestamp: current].
 - If there are now **three** buckets of size 2^j :
 - Merge the two oldest (leftmost) buckets of size 2^j into one bucket of size 2^{j+1} .
 - Set its timestamp to the more recent of the two merged buckets.
 - Recursively check if this generates three buckets of size 2^{j+1} (analogous to ripple-carry addition).

DGIM Query Answering & Error Bounding

- **Query Answering for Window $k \leq N$:**

- Identify bucket b_{oldest} whose timestamp covers the k -th position.
- Sum the sizes of all newer buckets: $\sum_{\text{newer}} \text{size}$.
- Add half the size of b_{oldest} :

$$\widehat{C} = \frac{1}{2} \text{size}(b_{\text{oldest}}) + \sum_{b_i > b_{\text{oldest}}} \text{size}(b_i)$$

- **Proof of 50% Relative Error:**

- Let $\text{size}(b_{\text{oldest}}) = 2^j$.
- True count $C \geq 1 + \sum_{i=0}^{j-1} 2^i = 2^j$.
- Max absolute error occurs if only one 1 falls within the query window:

$$\text{Error} \leq \frac{1}{r} 2^{j-1} - 1 < \frac{1}{2} C$$

- **Generalization:** Allowing up to r buckets of each size bounds relative error to $\leq 1/(r-1)$.

Questions?

—